PLATEFORME FINANCE & TRÉSORERIE

Comment SIMTIS fonctionne, techniquement

Du clic dans le navigateur à la ligne dans PostgreSQL : chaque mécanisme expliqué pas à pas

DATE
03/11/2026

ÉTAT DÉCRIT
Commit 982f379 · migration 0045

PUBLIC
Développeur, exploitation

Ce document complète l'« Architecture technique V2 » (21/10/2026) : au lieu de lister les composants, il suit les données à travers l'application (connexion, import d'un relevé, import Sage, rapprochement, frais, position, rapports, déploiement). Le fichier CLAUDE.md du dépôt reste la référence de chaque règle.

Sommaire

01 SIMTIS en une page

02 Les conteneurs et le trajet d'une requête

03 Le frontend Next.js

04 Le backend FastAPI et ses couches

05 Connexion, sessions et permissions

06 La base de données

07 Parcours 1 : importer un relevé bancaire

08 Parcours 2 : importer le grand livre Sage

09 Parcours 3 : le moteur de rapprochement

10 Parcours 4 : valider, rapprocher à la main, annuler

11 Parcours 5 : les frais bancaires et l'export Sage

12 Soldes, position bancaire et solde comptable

13 Dashboard, rapports et exports

14 Audit et historique

15 Tests, qualité et intégration continue

16 Production, déploiement automatique et sauvegardes

17 Flux des données entre les fichiers

18 Carte du code : où trouver quoi

1. SIMTIS en une page

SIMTIS remplace les fichiers Excel de la Trésorerie (position bancaire) et de la Comptabilité (rapprochement bancaire). C'est une couche d'intégration, de contrôle et de pilotage : les données viennent des banques et de Sage, SIMTIS les normalise, les rapproche et les présente. Il ne remplace pas Sage.



34	45	105	~1 200
tables PostgreSQL	migrations Alembic	points d'API	tests (≈ 950 backend, ≈ 260 frontend)

Trois principes qui expliquent tout le reste

Le serveur décide

Toutes les règles métier et tous les calculs financiers sont dans le backend. Le navigateur affiche et prépare. Il ne parle jamais à la base. Quand l'écran calcule quelque chose (un total coché, un solde recalculé), le serveur le recalcule à la confirmation et ne fait jamais confiance au chiffre envoyé.

L'argent est exact

`NUMERIC(18,2)` en base, `Decimal` en Python, chaînes de caractères dans le JSON (`"500000.00"`), centimes en `BigInt` dans le navigateur. Jamais de `float`. Jamais de somme de deux devises.

Proposer, jamais décider seul

Le moteur de rapprochement propose avec un score. Seul un humain disposant de la permission valide. Une proposition, même forte, laisse les lignes « À vérifier ».

Tout est tracé

Chaque action sensible écrit dans `audit_logs` (qui, quand, IP, avant, après) dans la **même transaction** que la modification. La table est en ajout seul, imposé par un déclencheur PostgreSQL.

Deux sociétés, jamais mélangées. Simtis et Tefil (code `SOCX`). La société active est choisie dans l'en-tête (`useCompany()`). Chaque écran et chaque requête portent un `company_id`. Le moteur ne rapproche jamais deux sociétés entre elles, et chaque modification verrouille la ligne de la société concernée.

2. Les conteneurs et le trajet d'une requête

Navigateur — Edge / Chrome des PC de la Trésorerie et de la Comptabilité

▼ HTTP (aujourd'hui) puis HTTPS · http://10.77.77.11

Caddy — Production seulement : `/api/*` → backend, le reste → frontend · corps jusqu'à 100 Mo, délai 10 min

▼

Frontend Next.js 16 — React 19, TypeScript, Tailwind 4 · port 3000 · build « standalone »

Backend FastAPI — Python 3.11, Uvicorn (2 workers en prod) · port 8000

▼ SQLAlchemy 2 + psycopg 3

PostgreSQL 17 — volume `db_data` · port 5434 sur le poste de développement

Exemple : la Comptabilité clique « Valider » sur une proposition

1. **Composant React.** Le bouton de `ProposalCard` déclenche `validateMatch(id)` dans `ProposalsTab`. Cette fonction vient de `frontend/services/reconciliation.ts` et passe par `apiFetch` (`frontend/lib/api.ts`).
2. **apiFetch** ajoute l'en-tête `Authorization: Bearer <jeton>` (gardé en mémoire uniquement), envoie `POST /api/reconciliation/matches/{id}/validate`. En production, l'URL est relative (`/api`) : même origine, Caddy aiguille.
3. **FastAPI** trouve la route dans `protected_router` (`app/api/router.py`). La dépendance `require_permission(RECONCILIATION_VALIDATE)` décode le JWT, vérifie que la session est encore ouverte, recharge les permissions **depuis la base**, puis répond 401 ou 403 si besoin.
4. **Endpoint** (`app/api/reconciliation.py`) : quelques lignes qui passent la main à `reconciliation_service.validate(db, id, acteur_id=…, ip=…)`.
5. **Service** : verrouille la société (`SELECT … FOR UPDATE`), verrouille la correspondance, vérifie qu'elle est « Proposée » et équilibrée (montants opposés au centime), attribue un code « BP-26-000123 » via un compteur sans trou, passe les lignes en « Rapprochée », écrit l'audit.
6. **Repository** : les lectures et écritures SQL (`reconciliation_repository`). Une seule validation (`commit`) à la fin. Si une règle échoue, le service lève `ConflictError("…")`, rien n'est écrit.
7. **Réponse** : `app/main.py` transforme `ConflictError` en `409 {"detail": "message en français"}`. `apiFetch` l'affiche tel quel. Sinon le schéma Pydantic `CorrespondanceOut` sérialise le résultat et l'écran se recharge.

Développement et production

	Développement (`docker-compose.yml`)	Production (`docker-compose.prod.yml`)
Backend	code monté, `--reload`, polling des fichiers (Windows)	`Dockerfile.prod` : sans dépendances de dev, utilisateur non root, 2 workers, `--proxy-headers`
Frontend	`next dev --webpack` (Turbopack ne voit pas les changements sous Windows)	`next build --webpack`, sortie `standalone`, `NEXT_PUBLIC_API_URL=/api`
Ports publiés	3000, 8000, 5434 (127.0.0.1 par défaut)	80 (et 3000) via Caddy uniquement

	Développement (`docker-compose.yml`)	Production (`docker-compose.prod.yml`)
Démarrage du backend	`alembic upgrade head` → `python -m app.seeds` → `uvicorn` : le schéma et les données de référence sont toujours à jour avant de servir.	

3. Le frontend Next.js

Une application React organisée en quatre étages : les pages (routes), les composants (affichage), `services/` (appels à l'API) et `lib/` (règles pures testées, client HTTP).

Dossier	Contenu
`app/(app)/…`	Les 11 routes protégées : dashboard, banques, comptes, releves, ecritures, rapprochement, position-bancaire, rapports, historique, administration, ecarts (masquée), plus `/login` et `/design-system`. `app/(app)/layout.tsx` monte `AppShell` (menu, en-tête, société active).
`components/<module>/`	Un dossier par écran (rapprochement, releves, ecritures, position, dashboard…) et `components/ui/` pour les briques communes : `Card`, `Button`, `DataTable`, `Modal`, `StatusBadge`, `Pagination`, `Field`…
`services/*.ts`	Une fonction par appel d'API (`analyseStatement`, `confirmStatement`, `runReconciliation`, `validateMatch`, `getBanquesTable`, `validerFrais`…). Elle construit l'URL et le corps, et passe toujours par `apiFetch` ou `apiDownload`.
`lib/*.ts`	Fonctions pures sans React, chacune avec son fichier `*.test.ts` (Vitest) : formats (`format.ts`), totaux en centimes (`importLines.ts`, `statementLines.ts`), panier de rapprochement (`reconciliation.ts`), formules des cellules (`formule.ts`), permissions (`permissions.ts`).
`types/`	Les types TypeScript des réponses de l'API, alignés sur les schémas Pydantic.

Comment l'écran parle à l'API

- `apiFetch` est le seul point d'appel. Il ajoute le jeton, transforme un 422 sans message métier en « Requête refusée : données invalides. », tente **une** fois `/api/auth/refresh` sur un 401, puis renvoie vers `/login` si la session est finie.
- `apiDownload` télécharge les exports (`.xlsx`, `.csv`, `.pdf`) avec le même jeton.
- Les fichiers importés partent en `multipart/form-data`. Les lignes corrigées d'un relevé partent dans une **partie fichier JSON** (`lignes`), car un champ texte multipart est limité à 1 Mio par Starlette.
- Sans `NEXT_PUBLIC_API_URL`, l'adresse de l'API est l'hôte de la page sur le port 8000 (`apiUrlFor`). Un téléphone du réseau local fonctionne donc sans configuration.

L'état de l'écran

- Pas de bibliothèque d'état globale : état local React, contexte pour la société active (`CompanyProvider`) et l'utilisateur.
- Préférences d'affichage dans `localStorage`, lues avec `useSyncExternalStore` et toujours protégées par `try/catch` : menu réduit (`simtis.menu.reduit`), période du rapprochement, carte « Solde comptable » repliée. Jamais de jeton ni de donnée métier.
- Les filtres partageables passent dans l'URL (onglet, banque et période du dashboard).

Calculs à l'écran, revérifiés par le serveur

Certaines valeurs sont calculées en direct pour que l'utilisateur voie l'effet de ses choix : solde après import, total des lignes cochées, écart du panier, ventilation HT / TVA, formules `=1200,50+350*2` des cellules Prévisions. Tous

ces calculs travaillent en **centimes BigInt** (aucune erreur d'arrondi). Le serveur refait le calcul à la confirmation et refuse si le résultat ne colle pas.

Design system

Palette teal en jetons CSS (`app/globals.css`, copiée dans le skill `simtis-design`), icônes Lucide, vrais logos (`public/logo-simtis.png`, `public/banques/*.png` via `BankLabel`), graphiques Recharts. Les montants passent tous par `formatAmount()`, qui ne tronque jamais.

4. Le backend FastAPI et ses couches

api/	Endpoints minces : lire la requête, exiger une permission, appeler un service
schemas/	Pydantic : validation d'entrée (`extra="forbid"`) et forme des réponses
services/	Toutes les règles métier, l'orchestration, l'audit, la transaction
repositories/	Requêtes SQL (SQLAlchemy), verrous, lectures en lot
models/	Tables ORM, contraintes, index

Un endpoint réel

```
# app/api/banks.py
can_manage = require_permission(PermissionCode.BANKS_MANAGE)

@router.post("", response_model=BankOut, status_code=201)
def create_bank(body: BankCreate, request: Request,
                db: Session = Depends(get_db),
                user: CurrentUser = Depends(can_manage)) -> BankOut:
    summary = bank_service.create_bank(db, code=body.code, nom=body.nom, ...,
                                       acteur_id=user.id, ip=client_ip(request))
    return BankOut.from_summary(summary)
```

Aucun `if` métier dans l'endpoint. Le service fait les contrôles, écrit, audite et valide la transaction.

Les erreurs

```
# un service
raise ConflictError("Ce compte a déjà un relevé : son solde vient du relevé.")
# app/main.py : NotFoundError → 404, ConflictError → 409, réponse {"detail": message}
```

Le message, en français, va tel quel à l'utilisateur. Un champ inconnu ou interdit dans le corps donne 422 (Pydantic), jamais un oubli silencieux.

Transactions et verrous

- **Une requête = une session SQLAlchemy** (`get_db`). Le service fait un seul `commit` à la fin. En cas d'exception, rien n'est écrit : c'est ce qui rend les opérations « tout ou rien » (validation en lot, suppression d'import, reprise).
- **Contrôler avant d'écrire.** Les opérations en lot vérifient toutes les lignes avant la première écriture et nomment la ligne fautive dans le 409.
- **Verrou de société.** Les actions de rapprochement verrouillent la ligne `companies` : deux validations simultanées ne peuvent pas prendre la même ligne. Le lancement du moteur utilise `FOR UPDATE NOWAIT` : un deuxième lancement reçoit tout de suite « Un rapprochement ou une validation est déjà en cours » au lieu d'attendre.
- **Compteurs sans trou** (codes de rapprochement, N° de pièce des frais) : `INSERT … ON CONFLICT … DO UPDATE … RETURNING`, atomique en une requête.

Règles pures

Les calculs sont isolés dans des fonctions sans base de données, faciles à tester : `position_service` (formules du CDC), `reconciliation_scoring` (score), `reconciliation_groupes` (groupes N → 1), `normalization_service` (lecture des dates, montants, libellés, hash), `frais_service` (familles de frais, TVA, lignes Sage), `rapports_service.etat_controle`. Les services d'orchestration lisent la base, appellent ces fonctions, puis écrivent.

Commandes en ligne (`python -m app.cli`)

`create-user`, `recalculer-soldes`, `reprise-rapprochement` (reprise d'un état Excel de rapprochement, en simulation par défaut, `--appliquer` pour écrire). Elles passent par les mêmes services et le même audit que l'API.

5. Connexion, sessions et permissions

Le mécanisme à deux jetons

1. **Connexion** (`POST /api/auth/login`) : e-mail normalisé, mot de passe vérifié en **Argon2** (`pwdlib`). Après 5 échecs, le compte est bloqué 15 minutes.
2. Le serveur crée une ligne `user_sessions` (durée 12 h) et renvoie :
   · un **jeton d'accès JWT** de 15 minutes, gardé **en mémoire** par le navigateur (jamais dans `localStorage`). Il contient l'utilisateur et l'id de session, **pas les permissions** ;
   · un **jeton de session** dans un cookie `HttpOnly` limité au chemin `/api/auth`. En base, seul son hash est stocké.
3. À chaque requête, `get_current_user` décode le JWT (60 s de tolérance d'horloge, car la VM Docker Desktop recule parfois de ~26 s), vérifie que la session est encore ouverte et l'utilisateur actif, puis **recharge ses permissions depuis la base**.
4. Quand le jeton d'accès expire, `apiFetch` appelle `/api/auth/refresh` : le cookie prouve la session et un nouveau JWT est émis.
5. Déconnexion, désactivation ou réinitialisation du mot de passe ferment la session : l'accès est coupé **immédiatement**, même avec un JWT encore valide.

Permissions effectives

```
permissions = permissions des rôles de l'utilisateur
            + ajouts personnels (user_permissions.accordee = true)
            − retraits personnels (user_permissions.accordee = false)
```

Rôles : Administrateur, Trésorerie, Comptable, Responsable, Direction / Consultation, plus des rôles personnalisés (`PERSO_…`). Les codes vivent dans `app/core/permissions.py` (par exemple `statements.import`, `accounting.import`, `reconciliation.view`, `reconciliation.validate`, `accounting.prepare`, `position.view`, `forecasts.manage`, `admin.users`, `admin.roles`, `admin.settings`, `audit.view`). `frontend/lib/permissions.ts` en est une copie, vérifiée par un test.

> **Le menu ne protège rien.** Le frontend masque les pages selon les permissions (`ROUTE_PERMISSIONS`), mais la vraie barrière est le backend. Tout nouveau routeur est ajouté à `protected_router` (401 sans jeton), et chaque endpoint exige sa permission (403). `tests/test_permissions.py` liste les routes publiques et échoue si une autre route répond sans jeton.

Garde-fous de l'administration

Toujours au moins un administrateur actif (verrou sur le rôle ADMIN). Un administrateur ne peut ni se désactiver, ni perdre son propre rôle ADMIN, ni retirer `admin.users` / `admin.roles` au rôle ADMIN. Un nouvel utilisateur reçoit un mot de passe à usage unique généré par le serveur, affiché une seule fois.

6. La base de données

Les 34 tables par domaine

Domaine	Tables principales
Référentiel	`companies`, `banks`, `bank_accounts` (devise, type, LIGNE = `credit_autorise`, taux, journaux Sage, comptes comptables, solde comptable d'ouverture), `currencies`, `pointage_types` (74 catégories)
Soldes et saisies	`bank_account_balances` (historique par jour, source « Saisie » ou « Relevé »), `saisies_devises`, `saisies_previsions`, `saisies_previsions_jour`
Imports	`import_batches` (un par fichier, statut Confirmé / Annulé), `column_mappings` (modèles de colonnes), `bank_statements`, `bank_transactions`, `balance_checks`, `accounting_entries`
Rapprochement	`reconciliation_matches` (type 1-1, 1-N, N-1, N-N ; statut ; code ; score ; `detail_score` JSON), `reconciliation_match_items` (les lignes, avec `actif`), `reconciliation_rules` (grille du score), `reconciliation_code_counters`
Frais	`frais_piece_counters`, `frais_lignes` (lignes modifiées à la main), `frais_jours_ecartes`
Écarts (masqués)	`discrepancies` et leur historique
Sécurité et audit	`users`, `roles`, `permissions`, `role_permissions`, `user_roles`, `user_permissions`, `user_sessions`, `audit_logs`

Les relations clés

```
Société 1—N Compte bancaire —N—1 Banque
Compte 1—N Relevé (bank_statements) 1—N Opération (bank_transactions)
Compte 1—N Écriture Sage (accounting_entries)
Opération N—N Écriture  via  reconciliation_matches + reconciliation_match_items
```

Ce que la base garantit elle-même

- **CHECK** : sens des montants (`devise_meme_sens`), TVA cohérente, une famille de frais seulement pour les écritures SIMTIS, format des modèles de colonnes (`format_si_banque`), date du solde comptable…
- **Index uniques partiels** : une seule correspondance **active** par opération et par écriture (`reconciliation_match_items.actif`) : il est physiquement impossible de rapprocher deux fois la même ligne. Un seul code par société, un seul modèle de colonnes par banque × format × « en cours ».
- **Déclencheurs** : `audit_logs` refuse `UPDATE` et `DELETE` ; `bank_accounts_journaux_uniques` empêche un journal Sage d'appartenir à deux comptes actifs.

Migrations et données de référence

- **Alembic**, 45 migrations (0045 = un modèle de colonnes par format). Une migration autogénérée est toujours relue : elle oublie les déclencheurs et les fonctions. `alembic check` ne compare pas les CHECK : chaque contrainte a son propre test dans `tests/test_constraints.py`.
- Certaines migrations ne touchent que les données (re-pointage, correction du sens, clôture des écarts) et écrivent leur audit.

- **Seeds** (`python -m app.seeds`) : banques, sociétés, rôles, permissions, catégories, règles du score. Ils **créent sans jamais modifier** : une valeur changée par un utilisateur survit au redémarrage. Les données de démonstration (`--demo`) sont refusées en production.

7. Parcours 1 : importer un relevé bancaire

Écran `/releves`, assistant Fichier → Validation. Deux appels : `/analyse` (n'écrit rien) puis `/confirm` (écrit tout en une transaction). Le serveur est sans état entre les deux : la confirmation renvoie et réanalyse le fichier.

Étape A : analyse (`POST /api/statements/import/analyse`)

1. **Lecture du fichier** (`import_file.read_sheet`) : le format est reconnu par le **contenu**, pas par l'extension. Signature OLE2 → `.xls` avec `xlrd` ; `%PDF` → `pdfplumber` (tableaux tracés, sinon colonnes déduites de la position des mots, en-têtes répétés retirés) ; sinon `.xlsx` avec `openpyxl`. Limites : 20 Mo, 50 000 lignes, 500 pages.
2. **Ligne d'en-tête et colonnes.** SIMTIS cherche la ligne d'en-tête, puis reconnaît chaque colonne par synonymes et abréviations (« DT valeur » → date de valeur, « Mnt » → montant). Le **modèle mémorisé** de la banque pour ce format (Excel ou PDF) et ce type de fichier (relevé ou opérations en cours) est appliqué en premier, puis complété par la détection. Si rien n'est reconnu, l'écran ouvre le mapping manuel.
3. **Lecture de chaque ligne** (`_read_line`, `normalization_service`) : dates, débit / crédit ou montant signé, libellé nettoyé, référence. Les lignes SOLDE INITIAL / FINAL, totaux et titres sont **ignorées**, mais elles donnent le solde d'ouverture ou de clôture du fichier.
4. **Sens sans signe.** Quand la colonne Montant n'a pas de signe, le sens de chaque ligne est déduit de la chaîne des soldes. Une ligne non prouvée est une erreur « Sens introuvable », jamais un crédit par défaut.
5. **Hash de ligne** : empreinte du compte, des dates, du libellé, des montants, du solde et du rang (deux lignes identiques d'un même fichier restent deux opérations). Les hash déjà en base, cherchés par lots de 5 000, font les **doublons** : une ligne déjà importée n'est jamais cochable.
6. **Soldes** : solde lu dans le fichier, sinon calculé (solde précédent − débit + crédit dans l'ordre chronologique) à partir d'un solde d'ouverture pris dans cet ordre : ligne SOLDE INITIAL, clôture du relevé précédent, tableau Banques, ou saisie (premier import seulement). Un solde de banque présent sur certaines lignes est contrôlé et relance le calcul.
7. **Pointage** : valeur du fichier, sinon la **mémoire** (pointage le plus fréquent des opérations de même libellé normalisé), sinon le mot-clé le plus long du libellé parmi les 74 catégories, sinon « À choisir ».
8. **Opérations en cours** : libellé finissant par « (*) », ou fichier séparé coché « opérations en cours ». Pour AWB, une ligne définitive de même montant, même libellé sans « (*) », à 10 jours au plus, **complète** l'opération en cours au lieu de créer une deuxième ligne.

La réponse contient les lignes avec leur statut (Valide, Erreur, Doublon), les lignes ignorées, les soldes proposés et les compteurs des tuiles.

Étape B : validation à l'écran

`EditablePreview` affiche le relevé tel qu'il sera enregistré, au format standard en 11 colonnes, 100 lignes par page. L'utilisateur corrige une ligne, la décoche, ou change le solde d'ouverture (recalcul en direct, `runningBalances`). Aucune ligne ne peut être ajoutée. Une ligne en erreur cochée bloque la confirmation.

Étape C : confirmation (`POST /api/statements/import/confirm`)

1. Le fichier est renvoyé avec les lignes cochées et corrigées (partie JSON `lignes`, jusqu'à 60 Mo). Le serveur **réanalyse** et revérifie chaque ligne avec les mêmes règles. Il ne croit jamais l'écran.

2. Dans une seule transaction : `import_batches`, `bank_statements`, `bank_transactions` (origine Fichier / Corrigée, rang `ordre`), modèle de colonnes de la banque pour ce format, `balance_checks`, solde du jour de clôture (source « Relevé »), audit de chaque correction.
3. Le relevé s'ajoute au **relevé continu** du compte : la page lit toutes les opérations du compte dans l'ordre chronologique (date, relevé, `ordre`) et exporte le format standard en `.xlsx`.

Défaire un import. « Supprimer cet import » est tout ou rien : refusé si une ligne est dans un rapprochement validé (le 409 nomme les codes à annuler), et seulement pour le **dernier** relevé du compte. Le lot reste avec le statut « Annulé », et le même fichier peut être réimporté.

8. Parcours 2 : importer le grand livre Sage

Écran `/ecritures`, même pipeline que les relevés (`import_file` partagé), avec `accounting_import_service`.

1. **Lecture** du fichier `.xlsx` / `.xls` (un PDF Sage est refusé). Le modèle de colonnes est mémorisé par société, complété par la détection.
2. **Filtre des lignes** : on ne garde que la **ligne banque d'un journal bancaire**. Le journal doit être l'un des « Journaux Sage » d'un compte (par exemple 5243 et 5543 pour BP) et le compte doit commencer par l'un de ses comptes comptables (jusqu'à 9). Les contreparties et les autres journaux sont comptés comme ignorés.
3. **Montants** : débit / crédit restent comme dans Sage (un débit Sage est positif dans `montant`). Pour un compte EUR / USD, la colonne « Devise » est obligatoire et donne `montant_devise`, du même signe.
4. **Doublons** : par hash de ligne et par société. Une écriture de frais créée par SIMTIS qui revient de Sage (même compte, jour, montant, N° pièce ou libellé) est reconnue comme doublon.
5. **Solde comptable après import** : l'analyse calcule, pour chaque compte du fichier, le solde comptable avant import à la dernière date du fichier. L'écran y ajoute les lignes cochées (`soldesApresImport`, en centimes). La Comptabilité compare avec Sage avant de confirmer.
6. **Confirmation** avec `lignes_choisies` (numéros des lignes cochées). Une ligne en erreur ou déjà importée cochée donne 409. Aucune correction dans SIMTIS : une erreur se corrige dans Sage.

Écritures saisies à la main. « Ajouter une écriture » (`POST /api/accounting/entries`, motif obligatoire) crée une ligne d'origine « Manuelle », traitée comme une ligne importée. Même principe côté banque avec « Ajouter une opération », rangée dans le relevé « Saisies manuelles » du compte.

9. Parcours 3 : le moteur de rapprochement

« Lancer le rapprochement » (`POST /api/reconciliation/run`) compare les opérations bancaires libres aux écritures Sage libres d'une société, sur une période de 5 ans au plus, et enregistre des **propositions**.

1. Qui peut être comparé à qui

- Même **compte bancaire**. Jamais deux sociétés.
- **Montant exactement opposé** : un crédit banque est un débit Sage (`tx.montant == -montant_rapprochement(écriture)`, le montant en devise pour EUR / USD). Un montant différent n'est jamais proposé.
- **Même type d'opération**, lu au début du libellé : Chèque (Sage `EAR1°`, banque `ENCAISSEMENT CHEQUE`), Effet (`EAR°`, `LCN`, `REMISE D'EFFETS`), Espèces (`ESP`, `VERSEMENT DEPLACE`), Virement, Impayé, ou Autre.
- **Fenêtre de dates** : ± 10 jours (180 jours pour un chèque ou un effet de même numéro). Chèque et Effet comparent la date Sage à la date d'opération ; Espèces et Virement comparent l'échéance Sage à la date de valeur ou d'opération, la plus proche des deux.
- Sont exclus : lignes déjà rapprochées, opérations en cours, frais bancaires (ils passent par l'écran Frais), paires déjà rejetées depuis le dernier lancement.

2. Le score (sur 100), grille dans `reconciliation_rules`

Critère	Points	Règle
Montant	60 / 40	Montant opposé exact : 60 s'il a des centimes, 40 s'il est rond.
Date	30	Dégressif sur 3 jours (30 · 22,5 · 15 · 7,5 · 0). Espèces : barème fixe 30 / 15 / 10 / 5 jusqu'à 10 jours.
Libellé	20	Ressemblance des libellés normalisés (`cydifflib`, difflib compilé).
N° chèque / référence	40	Clés de pièce, ou numéro commun de 5 chiffres et plus (« CHEQUE N 0173813 » ↔ « EAR1° AXK 173813 »).
Bonus sans référence	+10	Espèces, Virement, Autre : la référence vaut 0 et chaque critère actif gagne 10.

Proposition à partir de **50**, « forte » à partir de **80**. Les seuils et les points se modifient dans Administration > Paramètres, et servent au lancement suivant.

3. Attribuer sans conflit

1. Toutes les paires possibles sont triées du score le plus haut au plus bas (`proposer`). Une ligne déjà prise laisse sa place à son candidat suivant.
2. **Égalité** au meilleur score d'une ligne : pas de proposition, les lignes sont « ambiguës » et listées dans l'onglet Propositions, où l'utilisateur choisit.
3. Un deuxième candidat à moins de 10 points est gardé dans `detail_score.second` : badge « Concurrente proche », jamais pré-coché.
4. **Groupes N → 1** (`reconciliation_groupes`) : une opération principale + une commission + sa TVA de 10 % au centime, même jour, dont la somme est l'opposé d'une écriture.
5. Les anciennes propositions du moteur sont remplacées. Les lignes proposées passent « À vérifier » en base (affichées « Non rapprochée » avec le badge « En proposition »).

4. Pourquoi il est rapide

Une année de Simtis représente environ 15 millions de paires dans les fenêtres de dates. Le moteur prépare chaque ligne une seule fois (texte, mots, numéros), indexe les écritures par (compte, montant opposé), écarte une paire dès qu'une borne prouve qu'elle ne peut pas atteindre 50 (`_candidat` : points sans libellé, puis bornes rapides du ratio), et ne calcule le ratio exact que pour les paires restantes. Lancement complet sur une année : environ 1 min 40 s au lieu de dizaines de minutes.

5. Recherche élargie

« Recherche élargie (1 an) » (`run-elargie`) cherche, au-delà de la fenêtre normale et jusqu'à 365 jours, les paires de montant opposé exact avec un numéro commun (ou, pour Espèces / Virement / Autre, un libellé ressemblant à 70 %). Une paire n'est proposée que si chaque côté n'a que ce candidat. Badge « Écart de N jours ».

10. Parcours 4 : valider, rapprocher à la main, annuler

Action	API	Ce que fait le serveur
Valider une proposition	`POST …/matches/{id}/validate`	Vérifie « Proposée » et l'équilibre au centime, attribue le code `BANQUE-AA-NNNNNN` (compteur par société, banque et année), lignes « Rapprochée », audit.
Valider la sélection	`validate-batch`	Jusqu'à 10 000 propositions, tout ou rien. L'audit note `en_lot` et `forte`.
Rejeter	`reject`, `reject-batch`	Lignes « Non rapprochée ». La paire n'est plus proposée jusqu'au prochain « Lancer le rapprochement ».
Rapprocher à la main	`POST …/matches` avec `transaction_ids[]` + `ecriture_ids[]`	1 à 50 lignes par côté, un seul compte, totaux exactement opposés, lignes libres, types compatibles. Puis correspondance validée 1-1 / 1-N / N-1 / N-N, et relance du moteur autour des lignes (ou relance ciblée depuis une carte ambiguë).
Aucune correspondance	`…/aucune-ecriture`, `…/aucune-operation`	Enregistre chaque candidat comme paire rejetée, remet la ligne « Non rapprochée », libère l'autre côté s'il n'a plus de candidat.
Annuler un rapprochement validé	`DELETE …/matches/{id}`	Motif obligatoire. Le code est gardé dans l'historique et n'est jamais réutilisé.

L'écran `/rapprochement`

- Deux volets, Transactions bancaires et Écritures comptables, avec filtres dans les en-têtes de colonnes (comme Excel) et chargement de 50 lignes de plus au défilement.
- Le **panier** (`PanierPanel`) : cocher des lignes des deux côtés montre les totaux, l'écart en centimes, le type, et active « Rapprocher (n → m) » quand c'est équilibré. Règle pure `etatPanier` dans `lib/reconciliation.ts`.
- Compteurs : Rapprochées, Non rapprochées, À comptabiliser (frais), Propositions en attente. Onglets Propositions (cartes, filtres Fortes / À vérifier / Éloignées / Ambiguës) et Historique (décisions, export du lettrage).

La reprise d'un état Excel

Pour démarrer un compte qui avait déjà un rapprochement Excel : la commande `reprise-rapprochement` lit les deux listes de lignes encore **non** rapprochées à une date, les retrouve en base (libellé sans espaces ni ponctuation, montant, date à 3 jours ; pièce pour Sage), et met **toutes les autres** lignes du compte dans une seule correspondance N-N validée « Reprise ». C'est tout ou rien, et refusé si le total ne s'équilibre pas au centime. Faite en production pour BP, BMCE et AWB.

11. Parcours 5 : les frais bancaires et l'export Sage

Les frais (timbres, commissions, TVA, agios…) ne sont pas rapprochés un par un : SIMTIS les regroupe par jour et crée l'écriture que la Comptabilité passera dans Sage. C'est la seule exception à « les écritures viennent de Sage ».

1. **Détection** (`reconciliation_repository.frais_bancaire`) : pointage (COM, DROIT, TVA, FRAIS, AGIOS…) ou libellé (DROIT(S) DE TIMBRE, COMMISSION, TAXE SUR VALEUR AJOUTEE…). VIREMENT COMMERCIAL et REMISE DOCUMENTAIRE n'en sont jamais. Ces opérations forment le compteur « À comptabiliser ».
2. **Propositions calculées, jamais stockées** (`GET /api/frais`) : par compte et par jour, une écriture « DT D'ENREGISTREMENT » (timbres), « COM » (commissions) et « FRAIS » (frais avec TVA incluse, HT = montant ÷ 1,10 arrondi au centime). Un jour où Sage a déjà l'écriture n'est pas proposé. Un frais de plus de 200 DH va dans « Autres frais à passer à la main ».
3. **Validation** (`POST /api/frais/valider`, `accounting.prepare`) : le serveur recalcule tout. Pour chaque jour, il crée l'écriture SIMTIS (`origine = 'SIMTIS'`, N° pièce `BP2610123` = banque, année, mois, rang, sans trou) et une correspondance validée N → 1 avec les opérations. L'utilisateur peut modifier les lignes avant (le total doit rester égal) ou écarter un jour.
4. **Frais à la main** (`POST /api/frais/manuelle`) : libellé libre, date, TVA optionnelle (HT + TVA = total, vérifié en centimes).
5. **Export pour Sage** (`GET /api/frais/export`) : un CSV « import paramétrable » (séparateur « ; », Windows-1252, virgule décimale, dates JJ/MM/AAAA). Chaque ligne va au crédit de son compte de charge (61671, 61714000, 61473000, 34552142 pour la TVA, agios par banque…). Les écritures sont marquées exportées : on ne peut plus les supprimer.

« Exporter pour la banque » produit en plus la liste des frais de plus de 200 DH, pour les justifier auprès de la banque.

12. Soldes, position bancaire et solde comptable

D'où vient le solde d'un compte

- **Le relevé fait foi** : dès qu'un compte a une opération importée avec un solde, son solde = solde de la dernière opération, dans l'ordre `ORDRE_CHRONOLOGIQUE` (date, relevé, rang dans le fichier). Les soldes saisis sont alors ignorés et une nouvelle saisie est refusée.
- Sans relevé (CIH, BMCI aujourd'hui), le solde vient de `bank_account_balances`, saisi à la main par jour (une nouvelle saisie du même jour corrige, l'audit garde l'ancien).
- Les opérations en cours d'un fichier séparé s'ajoutent au solde jusqu'à l'arrivée de leur ligne définitive (`position_service.avec_en_cours`).
- Une valeur jamais saisie reste `null`, jamais 0. Plusieurs comptes d'une même banque et devise s'additionnent dans la cellule, avec le détail en infobulle.

Les formules (fonctions pures de `position_service.py`)

```
Crédit disponible       = Crédit autorisé (LIGNE) − Crédit utilisé
Position disponible     = Solde bancaire + Crédit disponible
facilité de caisse      = Solde du jour                        (décision du 02/11/2026)
DEPASSEMENT d'un jour   = TOTAL des soldes + TOTAL des LIGNES   (négatif = ligne dépassée)
Disponible Fc reel      = Solde du dernier jour + LIGNE ; TOTAL = DEPASSEMENT
```

La page `/position-bancaire`

1. **Banques** : `GET /api/position/banques` calcule une ligne par jour, du premier solde jusqu'à la date choisie, pour le compte courant MAD de chaque banque (AWB, BMCE, BP, CIH, BMCI). Un jour sans solde réutilise le dernier connu, affiché en gris. Aucun solde avant le 01/01/2000 (garde-fou contre une année mal tapée). Les 3 derniers jours sont visibles, les autres par défilement.
2. **Prévisions** : saisies à la main ligne par ligne, chaque ligne avec sa date. `PUT` remplace toutes les lignes de la société à partir du jour affiché, en une transaction.
3. **Devises** : EUR et USD = soldes des comptes en devise, jamais convertis ; Exp DH convertible saisi à la main.
4. **Évolution de la position** : courbe Recharts des 30 derniers jours, construite à partir de la même réponse, sans deuxième appel. Un jour sans TOTAL est un trou, jamais 0.
5. « Exporter en PDF » : même tableau Banques, A4 paysage, généré par `reportlab`.

Le solde comptable

```
solde comptable(date) = solde d'ouverture saisi une fois (grand livre Sage, débit = positif)
                      + débits − crédits des écritures du compte après la date d'ouverture,
jusqu'à la date
écart                 = solde du relevé − solde comptable       (dans la devise du compte)
```

Carte « Solde comptable » de `/ecritures` (`solde_comptable_service`). Sans solde d'ouverture, seuls les mouvements sont donnés, jamais un départ à 0.

13. Dashboard, rapports et exports

Dashboard (`GET /api/dashboard`)

Le dashboard réutilise les règles existantes au lieu de les réécrire : soldes de `position_banques_service`, frais de `frais_bancaire`, statuts du rapprochement. Deux onglets : **Suivi** (alertes calculées, tâches en attente, taux de rapprochement par banque, dernières actions du journal) et **Indicateurs** (solde total MAD, encaissements, décaissements avec variation par rapport à la période précédente, évolution, répartition des frais). Période (5 ans au plus), banque et onglet sont dans l'URL. La cloche de l'en-tête compte les alertes du mois et se recharge à chaque changement de page.

Rapports Excel (`/rapports`, `rapports_service`, `openpyxl`)

Rapport	Contenu
Position du jour	Le tableau Banques dans sa structure imposée, en Excel ou en PDF.
État de rapprochement	Une feuille par compte : opérations à gauche, code au milieu, écritures Sage à droite, un bloc par correspondance validée.
Lignes non rapprochées	Une feuille par compte, du plus ancien au plus récent, sur 10 ans au plus. Solde du relevé et solde comptable en tête, et en bas un bloc **Contrôle** : solde banque rectifié, solde compta rectifié, écart (`etat_controle`).
Lettrage	Chaque correspondance validée, ligne par ligne, groupée par code, avec ligne de total.

Chaque export écrit une entrée d'audit `export_rapport`. Les dates Excel sont de vraies dates et les montants de vrais nombres.

14. Audit et historique

```
# dans un service, dans la même transaction que la modification
audit_service.log(db, user_id=user.id, action="validation_rapprochement",
                  entite="reconciliation_match", entite_id=match.id,
                  avant=avant, apres=apres, ip=client_ip(request))
db.commit()   # la modification et sa trace partent ensemble, ou aucune des deux
```

- `audit_service` masque les mots de passe et les jetons, et stocke les `Decimal` en texte exact.
- La table `audit_logs` est protégée par un déclencheur : aucune mise à jour ni suppression, même par erreur de code.
- `client_ip` lit l'IP réelle derrière Caddy (`X-Forwarded-For`, grâce à `--proxy-headers`).
- Écran `/historique` (`audit.view`) : filtres par utilisateur, action, entité et période, 50 lignes par page. Le détail avant / après champ par champ est chargé seulement quand on déplie une ligne (une entrée d'import peut peser 77 Ko). Les codes d'action ont un libellé français (`historique_service.ACTIONS`).

Exemples d'actions tracées : `import_releve`, `modification_operation`, `rapprochement_lance`, `validation_rapprochement`, `rapprochement_manuel`, `annulation_rapprochement`, `comptabilisation_timbres`, `suppression_import`, `reprise_rapprochement`, `modification_permissions`, `export_rapport`.

15. Tests, qualité et intégration continue

Backend (pytest)

- Base dédiée `simtis_test`, créée et migrée par `tests/conftest.py` : les tests exécutent les **vraies migrations**.
- Chaque test tourne dans une transaction annulée à la fin.
- Tests d'API de bout en bout avec `httpx` (import d'un vrai `.xlsx`, d'un `.xls` fabriqué avec `xlwt`, d'un PDF).
- `test_constraints.py` : une contrainte, un test qui échoue si la règle disparaît (`assert_rejected`).
- `test_migration_matches_models` : les modèles et la base migrée sont identiques.
- Lint et format : `ruff`.

Frontend (Vitest)

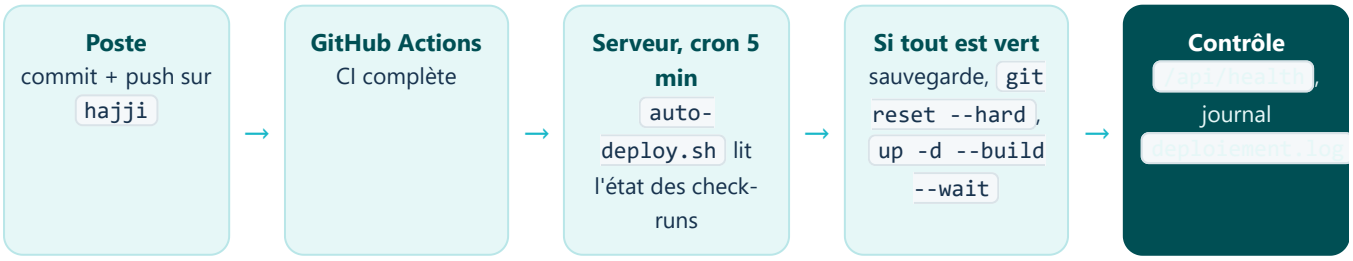
- Toutes les règles de `lib/` sont testées : centimes, panier, formules, filtres, périodes, permissions.
- `permissions.test.ts` vérifie que la copie des codes est identique au backend.
- ESLint, Prettier (`format:check`), `tsc` (`typecheck`) et `next build`.

Intégration continue

`.github/workflows/ci.yml` lance à chaque push les mêmes vérifications que le poste de développement : ruff, pytest avec PostgreSQL, lint, format, typecheck, Vitest et build du frontend. **Le déploiement automatique attend que tous ces contrôles soient verts.**

```
docker compose run --rm backend ruff check .  &&  docker compose run --rm backend pytest
cd frontend && npm run lint && npm run format:check && npm run typecheck && npm test && npm run build
```

16. Production, déploiement automatique et sauvegardes



- **Serveur** 10.77.77.11, dossier `~/simtis` : un clone git de la branche `hajji`, lu avec une clé de déploiement en lecture seule. Rien n'entre depuis l'extérieur. On ne modifie jamais le code sur le serveur.
- Au redémarrage, le backend applique les nouvelles migrations (par exemple 0045) et les seeds **après** la sauvegarde faite par le script.
- Un commit dont la CI ou le déploiement échoue est noté dans `.deploiement-echoue` et n'est pas retenté. Un verrou `flock` empêche deux déploiements simultanés.
- **HTTP pour l'instant** (`Caddyfile.http`, `APP_ENV=development` pour que le cookie de session ne soit pas « Secure »), en attendant un nom de domaine. Le passage en HTTPS (`tls internal`) consiste à retirer trois lignes de `.env`.
- **Sauvegardes** : `~/sauvegarde-simtis.sh` chaque nuit à 2 h (crontab), conservées 30 jours dans `~/backups`, plus une sauvegarde avant chaque déploiement et avant chaque intervention manuelle en production (reprises, corrections de libellés).

Interdit. `docker compose down -v` : supprime le volume `db_data`, donc toute la base. Le même nom de projet et le même volume servent en développement et en production.

17. Flux des données entre les fichiers

Pour chaque traitement important : les fichiers traversés, dans l'ordre, la fonction appelée dans chacun et la donnée qui passe au fichier suivant. Chemins relatifs à `frontend/` et `backend/app/`.

17.1 Le trajet commun à toutes les requêtes

#	Fichier	Rôle et fonction	Donnée transmise au suivant
1	`app/(app)/<page>/page.tsx`	Route Next.js, monte la vue du module (`RelevesView`, `RapprochementView`…)	société active (`useCompany()`)
2	`components/<module>/*.tsx`	Saisie et clics de l'utilisateur, état React	paramètres typés (id, période, fichier)
3	`services/<module>.ts`	Une fonction par point d'API : construit l'URL, la requête, le `FormData`	chemin + corps
4	`lib/api.ts`	`apiFetch` / `apiDownload` : jeton Bearer, rafraîchissement sur 401, messages d'erreur	requête HTTP (JSON ou multipart)
5	`main.py` → `api/router.py`	Application FastAPI, CORS, `protected_router` qui regroupe les routeurs des modules	requête routée
6	`api/deps.py` + `core/security.py`	`get_current_user` (JWT, session), `require_permission`	`CurrentUser` (id, permissions)
7	`schemas/<module>.py`	Validation Pydantic du corps (`extra="forbid"`)	objet d'entrée validé
8	`api/<module>.py`	Endpoint mince, appelle le service	arguments Python + `acteur_id`, `ip`
9	`services/<module>_service.py`	Règles métier, verrous, audit, `commit`	requêtes et lignes ORM
10	`repositories/<module>_repository.py`	SQL SQLAlchemy (`select`, `FOR UPDATE`, lots)	instruction SQL
11	`models/*.py` + `core/db.py`	Tables ORM, session (`get_db`)	SQL vers PostgreSQL
↩	`schemas/*Out` → JSON → `types/*.ts`	Retour : l'objet est sérialisé par le schéma de sortie, lu par le composant avec ses types TypeScript	affichage
!	`services/errors.py` → `main.py`	Erreur métier : `ConflictError` / `NotFoundError` → 409 / 404 `{"detail"}`, affichée par `apiFetch`	message en français

Fichiers d'aide utilisés partout : `services/audit_service.py` (trace), `core/permissions.py` (codes), `core/config.py` (réglages), `models/enums.py` (valeurs autorisées), `lib/format.ts` (affichage des montants et des dates).

17.2 Import d'un relevé : analyse

#	Fichier	Fonction	Donnée transmise
1	`components/releves/ImportWizard.tsx`	compte choisi dans `AccountPicker` + fichier	`ImportRequest`
2	`lib/statements.ts`	`fileProblem` (taille, type), `importForm`	`FormData` : fichier, `bank_account_id`, `mapping`, `en_cours`, `solde_ouverture`
3	`services/statements.ts`	`analyseStatement` → `POST /api/statements/import/analyse`	multipart
4	`api/statements.py`	`analyse_statement` (permission `statements.import`), `_content`, `_mapping`	octets du fichier, nom, options
5	`services/import_service.py`	`analyse_statement` : orchestration	appels ci-dessous
6	`services/import_file.py`	`check_file`, `format_fichier` (Excel / PDF), `read_sheet` (openpyxl, xlrd, pdfplumber → `lignes_pdf` / `lignes_par_mots`), `detect_header`, `columns_of`	lignes brutes (tuples) + colonnes
7	`repositories/import_repository.py`	`latest_mapping(banque, type, format, en_cours)` via `_saved_mapping`, puis `import_file.mapping_from_headers`	`Mapping` champ → colonne
8	`services/normalization_service.py`	appelé par `_read_line` : `parse_date`, `parse_amount`, `clean_libelle`, `extract_reference`, `balance_line_kind`, `est_en_cours`, `line_hash`	`AnalysedLine` (valeurs propres + erreurs)
9	`import_service` + `import_repository`	`Pointages.guess` (`pointages_of_company`), `_mark_duplicates` (`existing_line_hashes`, lots de 5 000), `_marquer_en_cours` (`operations_en_cours`, AWB), `_balances` / `running_balances` (`last_operation_before`, `last_known_balance`, `_opening_backwards`)	`StatementAnalysis` complète
10	`schemas/statement.py`	`AnalyseOut.from_analysis`	JSON : colonnes, lignes, ignorées, soldes, compteurs
11	`components/releves/EditablePreview.tsx`, `FilterTile`, `IgnoredLinesTable`	affichage et corrections, règles de `lib/statementLines.ts` (`runningBalances`, totaux en centimes) et `lib/importLines.ts` (`defaultChecked`, `pageOf`)	lignes cochées / corrigées

Rien n'est écrit en base pendant l'analyse. Seules des lectures ont lieu (modèle de colonnes, hash existants, soldes précédents, pointages connus).

17.3 Import d'un relevé : confirmation

#	Fichier	Fonction	Donnée transmise
1	`ImportWizard.tsx` → `services/statements.ts`	`confirmStatement` → `POST /api/statements/import/confirm`	même `FormData` + partie fichier JSON `lignes`
2	`api/statements.py`	`confirm_statement`, `_lines_text`, `_json_field(LignesSoumisesIn)`	lignes soumises validées
3	`import_service.confirm_statement`	réanalyse le fichier (étapes 5 à 9 ci-dessus), revérifie chaque ligne soumise avec `_read_line`, recalcule les soldes	lignes définitives
4	`import_service` → `import_repository.add`	crée `ImportBatch`, `BankStatement`, `BankTransaction` (`ordre`, `origine`), `BalanceCheck`, solde du jour « Relevé » ; `_save_mapping` ; `_completer_en_cours` pour AWB	objets ORM
5	`services/audit_service.py`	`log` : `import_releve`, corrections, `remplacement_en_cours`	lignes `audit_logs`
6	`models/imports.py`, `models/referentiel.py`	un seul `commit`	tables `import_batches`, `bank_statements`, `bank_transactions`, `column_mappings`, `balance_checks`, `bank_account_balances`
7	`AccountStatementCard.tsx` → `getAccountStatement`	`GET /api/statements/accounts/{id}` → `import_service.account_statement` → `import_repository.account_transactions` (`ORDRE_CHRONOLOGIQUE`)	relevé continu du compte

Effets en aval : les nouvelles opérations deviennent visibles pour `position_repository.dernier_solde_releve` (solde du compte, tableau Banques, cartes), pour `reconciliation_repository.free_transactions` (moteur) et pour `frais_repository.frais_libres` (frais à comptabiliser).

17.4 Import du grand livre Sage

#	Fichier	Fonction	Donnée transmise
1	`components/ecritures/AccountingImportWizard.tsx`	`analyseEntries` puis `confirmEntries` (`services/accounting.ts`, `lib/accounting.ts` `accountingForm`)	fichier + `company_id` (+ `lignes_choisies`)

#	Fichier	Fonction	Donnée transmise
2	`api/accounting.py`	`analyse_entries` / `confirm_entries` (`accounting.import`)	octets + options
3	`services/accounting_import_service.py`	`analyse_entries` : `import_file.read_sheet` (sans PDF), journaux et comptes de `account_service.journaux_sage` / `comptes_comptables` , `_read_entry` , `_mark_duplicates` , `_mark_simtis_entries` , `_summarise`	`EntryLine` + résumé par compte
4	`services/solde_comptable_service.py` → `repositories/accounting_repository.py`	`_soldes_avant_import` : ouverture du compte + `mouvements_comptables` jusqu'à la dernière date du fichier	`par_compte[].solde_avant`
5	`schemas/accounting.py` → `lib/importLines.ts`	`TotalCompteOut` ; à l'écran `totauxCoches` , `soldesApresImport` (centimes BigInt)	tuile « solde comptable après import »
6	`accounting_import_service.confirm_entries`	réanalyse, contrôle des lignes choisies, création `ImportBatch` + `AccountingEntry` , modèle de colonnes de la société, audit, `commit`	table `accounting_entries`

17.5 Lancer le rapprochement

#	Fichier	Fonction	Donnée transmise
1	`components/rapprochement/RapprochementView.tsx`	`runReconciliation(companyId, filter)` (`services/reconciliation.ts`)	`POST /api/reconciliation/run` : société, compte, période
2	`api/reconciliation.py`	endpoint `run` (`reconciliation.validate`)	arguments
3	`services/reconciliation_service.py`	`run` : `_company(sans_attendre=True)` → `reconciliation_repository.lock_company(nowait=True)` , `_check_period` , `grille` (`all_rules`), `lever_rejets`	société verrouillée, `Grille`
4	`reconciliation_service._engine` ↔ `reconciliation_repository`	`free_transactions` , `free_entries` , `rejected_pairs` , `fee_transaction_ids` (frais exclus), `automatic_proposals_of_period` (anciennes propositions)	lignes ORM → `_operation` / `_ecriture`

#	Fichier	Fonction	Donnée transmise
5	`services/reconciliation_scoring.py` (pur)	`proposer` : `types_compatibles`, fenêtres de dates, `_candidat` (bornes), `score`, attribution du plus haut score, égalités	propositions 1 → 1 + lignes ambiguës
6	`services/reconciliation_groupes.py` (pur)	`grouper` : principal + commission + TVA, `score_groupe`	propositions N → 1
7	`reconciliation_service._enregistrer_proposition` → `reconciliation_repository`	`delete_matches` (anciennes), `add` (`ReconciliationMatch` + `ReconciliationMatchItem`), `set_transaction_status` / `set_entry_status` (« À vérifier »), `reset_unmatched_to_check`	tables `reconciliation_matches`, `reconciliation_match_items`, statuts
8	`audit_service.log` + `_commit`	audit `rapprochement_lance` (compteurs, rejets levés)	résumé JSON
9	`ProposalsTab.tsx` → `listProposals`, `listAmbiguous` ; `TransactionsPane` / `EntriesPane` → `listTransactions`, `listEntries`	relecture : `list_matches` → `_views` (20 lignes max par côté), `list_transactions` → `transactions_query`, `count_by_status`	cartes, volets, compteurs (`lib/reconciliation.ts` : `filterProposals`, `defaultSelection`, `compteurStatut`)

17.6 Valider (une proposition, une sélection, un panier)

#	Fichier	Fonction	Donnée transmise
1	`ProposalsTab.tsx` / `MatchPanel.tsx` / `PanierPanel.tsx`	`validateMatch`, `validateMatches` (`aValider`), `matchManually` (`etatPanier`)	id(s) ou `transaction_ids[]` + `ecriture_ids[]`
2	`api/reconciliation.py`	`/matches/{id}/validate`, `/matches/validate-batch`, `POST /matches`	corps validé (`schemas/reconciliation.py`)
3	`reconciliation_service`	`validate` / `validate_batch` / `match_manually` : `_locked_match`, `_validate` + `_check_amounts` (`montant_rapprochement`), `_new_code`	contrôles passés
4	`reconciliation_repository.next_code_number`	`INSERT … ON CONFLICT … RETURNING` sur `reconciliation_code_counters` (société, banque, année)	code `BP-26-000123`
5	`reconciliation_service` → repository	`_decide` (statut Validée, `decide_par_id`), lignes « Rapprochée », `release_pending_matches` (propositions concurrentes rejetées), relance `_engine` ou `_relance_ciblee` (manuel)	lignes mises à jour

#	Fichier	Fonction	Donnée transmise
6	`audit_service.log` + `commit`	`validation_rapprochement` / `rapprochement_manuel`	`CorrespondanceOut` / `ValidationLotOut`

17.7 Tableau Banques de la position

#	Fichier	Fonction	Donnée transmise
1	`components/position/BanquesTable.tsx`	`getBanquesTable(companyId, jour)` (`services/position.ts`)	`GET /api/position/banques`
2	`api/position.py`	→ `position_banques_service.banques_table` (`position.view`)	société, date
3	`repositories/position_repository.py`	`active_banks`, `current_mad_accounts`, `comptes_alimentes_par_releve`, `last_operation_soldes` / `dernier_solde_releve` (relevé), `soldes_until` (saisies), `en_cours_hors_releve`	soldes bruts par compte et par jour
4	`position_banques_service._soldes`	choisit la source (relevé sinon saisie), ajoute les opérations en cours (`position_service.avec_en_cours`)	`BanqueColonne` / `CompteColonne`
5	`services/position_service.py` (pur)	`tableau_banques` : une ligne par jour, `somme_soldes`, `ligne_au`, TOTAL, DEPASSEMENT, Disponible Fc reel, dernier solde connu	`TableauBanques`
6	`schemas/position.py` → `BanquesTable` → `PositionView` (`onCharge`) → `PositionChart`	affichage du tableau, puis courbe avec `evolutionPoints` (`lib/position.ts`), sans deuxième appel	tableau + graphique

La même fonction `banques_table` alimente aussi le rapport Excel `rapports_service.position_du_jour` et le PDF `rapports_service.position_pdf` (reportlab). Le dashboard réutilise la même règle de soldes par `position_banques_service.soldes_par_compte` : une seule règle de calcul pour toutes les restitutions.

17.8 Frais bancaires : proposition, validation, export

#	Fichier	Fonction	Donnée transmise
1	`components/rapprochement/FraisCard.tsx`	`listFrais` (`services/frais.ts`) → `GET /api/frais`	société, période

#	Fichier	Fonction	Donnée transmise
2	`api/frais.py` → `frais_service.tableau`	`propositions` : `frais_repository.frais_libres` (filtre `reconciliation_repository.frais_bancaire`), `jours_dans_sage`, `regrouper` / `regrouper_timbres`, `ventilation`, `lignes_gl`, `compte_charge` ; `autres_frais` ; `ecritures` (déjà comptabilisées)	propositions calculées (non stockées)
3	`FraisCard` / `EditFraisModal`	sélection et modifications, contrôlées par `lib/frais.ts` (`etatLignes`, `ventilerTva`)	`validerFrais(companyId, jours)`
4	`frais_service.valider`	recalcule, `_lignes_valides`, `frais_repository.next_piece_number` → `numero_piece`, crée `AccountingEntry` (origine SIMTIS) + `frais_lignes`, puis `reconciliation_service.create_validated` (N → 1)	écriture + correspondance validée + audit
5	`RapprochementView` → `exporterFrais` → `frais_service.exporter`	lignes `lignes_ecriture`, `montant_csv`, encodage Windows-1252, marque `exportee_le`	fichier CSV pour Sage (`apiDownload`)
6	Retour par Sage	au prochain import Sage, `accounting_import_service._mark_simtis_entries` reconnaît l'écriture (N° pièce ou libellé)	ligne « Doublon », jamais importée deux fois

17.9 Vue d'ensemble : qui lit les données de qui

Table	Écrite par	Lue par
`bank_transactions`	`import_service`, `saisie_manuelle_service`, `reprise_service` (statuts), `reconciliation_service` (statuts)	position, rapprochement, frais, dashboard, rapports, solde comptable (écart)
`accounting_entries`	`accounting_import_service`, `frais_service` (SIMTIS), `saisie_manuelle_service`	rapprochement, frais (jours déjà dans Sage), `solde_comptable_service`, rapports
`bank_account_balances`	`balance_service` (saisie), `import_service` (clôture « Relevé »)	`position_repository`, `balance_service.figures_for_accounts` (cartes banques)
`column_mappings`	`import_service._save_mapping`, `accounting_import_service`	analyse suivante, Administration > Paramètres
`reconciliation_rules`	seeds, `parametres_service`	`reconciliation_service.grille` → moteur, groupes, recherche élargie

Table	Écrite par	Lue par
`reconciliation_matches` / `_items`	`reconciliation_service`, `frais_service`, `reprise_service`	écrans de rapprochement, historique, lettrage, rapports, dashboard, suppression d'import
`audit_logs`	`audit_service.log` (toutes les actions)	`historique_service` (`/historique`), activité du dashboard

18. Carte du code : où trouver quoi

Je cherche…	Backend	Frontend
Connexion, jetons	`core/security.py`, `services/auth_service.py`, `api/deps.py`	`lib/api.ts`, `services/auth.ts`, `components/auth/`
Lecture des fichiers	`services/import_file.py`, `normalization_service.py`	`components/imports/`
Import des relevés	`services/import_service.py`, `repositories/import_repository.py`	`components/releves/`, `lib/statementLines.ts`, `lib/statements.ts`
Import Sage, solde comptable	`accounting_import_service.py`, `solde_comptable_service.py`	`components/ecritures/`, `lib/importLines.ts`
Score du rapprochement	`reconciliation_scoring.py`, `reconciliation_groupes.py`	
Actions de rapprochement	`reconciliation_service.py`, `reconciliation_repository.py`	`components/rapprochement/`, `lib/reconciliation.ts`
Frais bancaires	`frais_service.py`, `frais_repository.py`	`FraisCard`, `lib/frais.ts`
Position, soldes	`position_service.py`, `position_banques_service.py`, `balance_service.py`	`components/position/`, `lib/position.ts`
Dashboard, rapports	`dashboard_service.py`, `rapports_service.py`	`components/dashboard/`, `components/rapports/`
Administration, audit	`admin_service.py`, `audit_service.py`, `historique_service.py`	`components/administration/`, `components/historique/`
Reprise d'un état Excel	`reprise_service.py`, `app/cli.py`	
Schéma de la base	`models/`, `alembic/versions/`, `app/seeds/`	`types/`
Déploiement	`docker-compose*.yml`, `backend/Dockerfile.prod`, `frontend/Dockerfile.prod`, `deploy/Caddyfile*`, `deploy/auto-deploy.sh`, `.github/workflows/ci.yml`	

Pour aller plus loin. `CLAUDE.md` à la racine décrit chaque règle et chaque décision datée. `docs/superpowers/specs/` contient les spécifications détaillées (relevés modifiables, tableau Banques, import Sage, rapprochement, écarts). Les PDF `docs/*_V4.pdf` et `*_V2.pdf` sont les versions pour le métier.